

Билет 6. Data race. Store buffering.

`std::atomic`. Spinlock. Реализация mutex через spinlock

0. Формулировка билета

Понятие data race. Store buffering. Класс `std::atomic` и его основные методы. Спинлок. Реализация mutex через спинлок.

Главная идея билета:

Если несколько потоков работают с общей памятью без синхронизации, программа может стать некорректной. Для простых разделяемых значений можно использовать `std::atomic`, который даёт атомарные операции. На атомиках можно построить простейший mutex — spinlock, где поток не засыпает, а активно крутится в цикле, пока блокировка не освободится.

Цепочка темы:

```
data race
↓
почему обычный ++counter ломается
↓
std::atomic
↓
RMW-операции: exchange, fetch_add, CAS
↓
spinlock через atomic exchange
↓
плюсы/минусы spinlock
↓
оптимизации: pause, TTAS, backoff
```

1. Что такое data race

Data race — это ситуация, когда:

1. два или более потока обращаются к одной и той же области памяти;
2. хотя бы одно обращение — запись;
3. между этими обращениями нет синхронизации / упорядочивания.

В C++ data race приводит к **undefined behavior**.

Простая формулировка для экзамена:

Data race — это несинхронизированный доступ нескольких потоков к одной памяти, где хотя бы один доступ является записью. В C++ это UB.

Пример:

```
int counter = 0;

void f() {
    ++counter;
}
```

Если несколько потоков одновременно вызывают `f`, будет data race.

2. Почему data race — это не просто “не тот ответ”

Иногда кажется:

ну максимум получим неправильное число

Но в C++ проблема глубже.

Если есть data race, стандарт языка не задаёт поведение программы.

Это значит:

программа может дать неправильный результат
может упасть
может пройти тесты
может сломаться после оптимизации компилятора
может вести себя по-разному на разных архитектурах

Компилятор имеет право оптимизировать код, предполагая, что data race в корректной программе нет.

Поэтому data race — это не “плохая межпоточность”, а именно **undefined behavior**.

3. Data race vs race condition

Эти понятия нельзя смешивать.

Data race

Низкоуровневая ошибка доступа к памяти.

одна ячейка памяти
несколько потоков
хотя бы одна запись
нет синхронизации

Это формальная ошибка C++.

Её может найти ThreadSanitizer.

Race condition

Более широкая логическая ошибка.

Результат зависит от порядка выполнения потоков.

Пример без data race:

```
std::atomic<bool> flag{false};

void try_set() {
    bool expected = false;

    if (flag.compare_exchange_strong(expected, true)) {
        // этот поток победил
    }
}
```

Data race нет: используется атомик.

Но есть race condition в широком смысле: кто первый выполнит CAS, тот победит.

Это не обязательно ошибка. Иногда это нормальная логика.

Коротко:

data race – ошибка синхронизации доступа к памяти
race condition – зависимость результата от порядка событий

4. Пример data race: ++counter

Код:

```
size_t counter = 0;

void worker(size_t iterations) {
    for (size_t i = 0; i < iterations; ++i) {
        ++counter;
    }
}
```

Если запустить несколько потоков, ожидаем:

```
counter == num_threads * iterations
```

Но без синхронизации результат часто будет меньше.

Почему?

Операция:

```
++counter;
```

не является одной неделимой операцией.

Она раскладывается примерно так:

1. load counter из памяти в регистр
2. add 1
3. store counter обратно в память

Сценарий потери обновления:

```
counter = 10

thread #1: load counter → 10
thread #2: load counter → 10

thread #1: add 1 → 11
thread #2: add 1 → 11

thread #1: store 11
thread #2: store 11
```

ожидали 12, получили 11

5. Исправление через mutex

Можно защитить счётчик мьютексом:

```
std::mutex m;
size_t counter = 0;

void worker(size_t iterations) {
    for (size_t i = 0; i < iterations; ++i) {
        std::lock_guard<std::mutex> lock(m);
        ++counter;
    }
}
```

Теперь data race нет.

Но для простого счётчика mutex может быть избыточен.

Нам не нужна сложная критическая секция. Нам нужна одна атомарная операция:

```
counter += 1
```

Для этого есть `std::atomic`.

6. `std::atomic`

`std::atomic<T>` — шаблонный класс для атомарных операций над объектом типа `T`.

Пример:

```
#include <atomic>

std::atomic<size_t> counter{0};

void worker(size_t iterations) {
    for (size_t i = 0; i < iterations; ++i) {
        counter.fetch_add(1);
    }
}
```

Теперь инкремент атомарен.

Несколько потоков могут одновременно делать:

```
counter.fetch_add(1);
```

и обновления не потеряются.

7. Что значит “атомарная операция”

Атомарная операция выглядит для других потоков как неделимая.

То есть нельзя увидеть её “посередине”.

Для `fetch_add(1)` можно думать так:

```
прочитать старое значение  
прибавить 1  
записать новое значение
```

но всё это происходит как один неделимый шаг относительно других атомарных операций над этим же объектом.

Важно:

Атомарность — это не значит, что физически операция мгновенная. Это значит, что в межпоточном наблюдении она выглядит как произошедшая целиком до или целиком после другой операции.

8. Основные методы `std::atomic`

`store`

Атомарная запись:

```
std::atomic<int> x{0};  
  
x.store(42);
```

`load`

Атомарное чтение:

```
int value = x.load();
```

exchange

Атомарно заменить значение и вернуть старое:

```
std::atomic<bool> flag{false};  
  
bool old = flag.exchange(true);
```

Смысл:

```
old = flag  
flag = true  
return old
```

но атомарно.

Именно `exchange` удобно использовать для простого spinlock.

fetch_add

Атомарно прибавить значение и вернуть старое:

```
std::atomic<int> counter{0};  
  
int old = counter.fetch_add(1);
```

После этого:

```
counter увеличился на 1  
old содержит старое значение
```

Есть похожие операции:

```
fetch_sub  
fetch_or  
fetch_and  
fetch_xor
```

`compare_exchange_weak` / `compare_exchange_strong`

CAS — compare-and-swap.

Идея:

если текущее значение равно `expected`,
заменить его на `desired`
иначе записать фактическое значение в `expected`

Пример:

```
std::atomic<int> x{10};  
  
int expected = 10;  
bool ok = x.compare_exchange_strong(expected, 20);
```

Если `x == 10`, то:

`x` станет 20
`ok == true`
`expected` останется 10

Если `x != 10`, например `x == 15`, то:

`x` останется 15
`ok == false`
`expected` станет 15

Это важно: `expected` обновляется.

9. `compare_exchange_weak` vs `strong`

Есть две версии CAS:

`compare_exchange_strong`
`compare_exchange_weak`

`strong`

Не должен давать ложный отказ.

Если значение действительно равно `expected`, операция должна пройти.

Обычно используется для одиночной попытки.

`weak`

Может дать **spurious failure** — ложный отказ.

То есть может вернуть `false`, даже если значение совпадало с `expected`.

Это связано с архитектурами типа ARM/POWER, где атомарность часто реализуется через LL/SC — load-linked/store-conditional.

Правило:

`weak` используют в цикле
`strong` используют для одиночной попытки

Типичный lock-free-паттерн:

```
T expected = old_value;

while (!atomic_value.compare_exchange_weak(expected, new_value)) {
    // expected уже обновлён фактическим значением
    // повторяем попытку
}
```

Для x86 часто разницы почти не видно, но писать надо переносимо.

10. RMW-операции

`exchange`, `fetch_add`, CAS — это **RMW-операции**.

RMW = read-modify-write.

То есть операция за один неделимый шаг:

читает старое значение
вычисляет новое
записывает новое

Примеры:

```
x.exchange(true);
counter.fetch_add(1);
head.compare_exchange_weak(expected, desired);
```

Почему RMW важны?

Потому что обычные `load` и `store` сами по себе не дают “проверить и установить” за один шаг.

Для mutex/spinlock нужна именно операция:

```
если свободно — занять
```

То есть:

```
проверить state
установить state = busy
```

и это должно быть атомарно.

11. Store buffering

Теперь важный пример из модели памяти.

Есть две общие переменные:

```
int x = 0;
int y = 0;
```

И две локальные:

```
int a = 0;
int b = 0;
```

Два потока:

```
// thread 1
x = 1;
a = y;

// thread 2
```

```
y = 1;  
b = x;
```

Интуитивно кажется, что результат:

```
a == 0 && b == 0
```

невозможен.

Почему кажется невозможным?

Если `a == 0`, значит thread 1 прочитал старое `y`, то есть будто бы thread 2 ещё не сделал `y = 1`.

Если `b == 0`, значит thread 2 прочитал старое `x`, то есть будто бы thread 1 ещё не сделал `x = 1`.

Кажется, оба потока не могли одновременно “не увидеть” запись другого.

Но на реальном железе такой результат возможен.

12. Почему `(0, 0)` возможен

Причина — **store buffer**.

У каждого ядра может быть буфер записи.

Когда поток делает:

```
x = 1;
```

запись может попасть не сразу в общую видимую память/кэш для других ядер, а сначала в store buffer текущего ядра.

Сценарий:

изначально `x = 0, y = 0`

thread 1 на ядре A:

`x = 1` уходит в store buffer ядра A
`a = y` читает `y` из памяти → 0

thread 2 на ядре B:

```
y = 1 уходит в store buffer ядра B
b = x читает x из памяти → 0
```

Итог:

```
a = 0
b = 0
```

При этом оба потока “записали” свои переменные, но записи ещё не стали видны другому ядру.

13. x86 TSO и store→load reorder

На x86 модель памяти называется TSO — Total Store Order.

Она довольно строгая, но разрешает один важный эффект:

более поздний load может обогнать более ранний store в другой адрес

Почему?

Потому что `store` записывает в store buffer, а следующий `load` может прочитать данные из памяти раньше, чем store стал виден другим ядрам.

Именно это и даёт `(0,0)` в тесте:

```
x = 1;
a = y;
```

`a = y` как будто обогнало видимость `x = 1` для другого ядра.

Важно:

```
store → load может переупорядочиться
store → store обычно нет
load → load обычно нет
load → store обычно нет
```

для x86 TSO.

14. Компиляторный барьер не лечит store buffering

Иногда пишут:

```
asm volatile("" ::: "memory");
```

Это компиляторный барьер.

Он запрещает компилятору переставлять операции через эту точку.

Но он не очищает store buffer процессора.

Поэтому такой код:

```
x = 1;
compiler_barrier();
a = y;
```

всё равно может дать `(0,0)`.

Чтобы исправить именно store buffering, нужен аппаратный барьер памяти, например `mfence` на x86, или атомики с подходящим memory order.

В полном разборе модели памяти это будет позже. В этом билете достаточно понимать:

Store buffering — аппаратный эффект. Компиляторный барьер его не убирает.

15. Атомики и store buffering

Если заменить обычные `int` на атомики с `seq_cst` по умолчанию:

```
std::atomic<int> x{0};
std::atomic<int> y{0};
```

и использовать:

```
x.store(1);
a = y.load();
```

то `std::memory_order_seq_cst` даёт более строгий порядок.

Тогда исход `(0,0)` становится невозможным.

Но важно:

Атомики — это не только “операция не разорвётся”. Они ещё участвуют в правилах видимости и упорядочивания между потоками.

Подробно `memory_order` будет в отдельном билете. Здесь можно сказать:

по умолчанию операции `std::atomic` имеют `memory_order_seq_cst`
это самый строгий порядок

16. Зачем нужен spinlock

Мы хотим реализовать примитив взаимного исключения:

```
lock();  
critical_section();  
unlock();
```

Свойство:

в каждый момент времени в `critical_section` находится не более одного потока

Это свойство называется:

`mutual exclusion`

`std::mutex` обычно при долгом ожидании паркует поток в ядре.

А spinlock работает иначе:

если `lock` занят, поток не спит,
а активно крутится в цикле,
пока `lock` не освободится

Отсюда название:

`spin` — крутиться

17. Safety и liveness

Для `mutex/spinlock` полезно различать два класса свойств.

Safety

Safety — “ничего плохого никогда не происходит”.

Для spinlock:

никогда два потока не находятся в критической секции одновременно

Это mutual exclusion.

Нарушение safety видно на конечном исполнении:

вот сейчас два потока внутри critical section

Liveness

Liveness — “что-то хорошее когда-нибудь происходит”.

Для spinlock:

если поток пытается захватить свободный lock, он когда-нибудь сможет войти

Нарушение liveness может проявиться как:

deadlock
livelock
starvation

Важно:

Safety должна выполняться при любом поведении планировщика.

Liveness обычно требует честности планировщика: если ОС никогда не даёт потоку CPU, никакой lock не гарантирует прогресс.

18. Виды прогресса

Global progress / deadlock-freedom

Хотя бы один поток из претендующих когда-нибудь продвинется.

система в целом не застряла

Но отдельный поток может голодать.

Local progress / starvation-freedom

Каждый поток, который пытается захватить lock, рано или поздно преуспеет.

нет голодания

Простой TAS spinlock обычно не гарантирует fairness:

один поток может постоянно проигрывать гонку за lock

19. Наивный spinlock через обычный bool

Попробуем:

```
bool state = false;

void lock() {
    while (state) {
    }

    state = true;
}

void unlock() {
    state = false;
}
```

Где:

```
state == false → свободно
state == true  → занято
```

Почему это ломается?

Проверка и установка — две отдельные операции.

Сценарий:

```
state = false

thread #1:
    проверил state == false
    ещё не сделал state = true
```



```
scheduler переключился

thread #2:
    проверил state == false
    сделал state = true
    вошёл в critical section

scheduler вернулся

thread #1:
    делает state = true
    тоже входит в critical section
```

Итог:

```
два потока внутри critical section
mutual exclusion нарушен
```

Кроме того, доступ к `state` из нескольких потоков без синхронизации — data race.

20. Нужна операция test-and-set

Проблема:

```
проверить state
установить state = true
```

должна быть одной атомарной операцией.

Это называется:

```
test-and-set
```

В C++ удобно использовать:

```
std::atomic<bool>::exchange
```

Она делает:

```
старое = state
state = true
return старое
```

атомарно.

21. Spinlock через `std::atomic<bool>::exchange`

```
#include <atomic>

class SpinLock {
public:
    void lock() {
        while (state_.exchange(true)) {
            // spin
        }
    }

    void unlock() {
        state_.store(false);
    }

private:
    std::atomic<bool> state_{false};
};
```

Как работает `lock`:

```
state_.exchange(true)
```

Если вернуло `false`:

```
раньше было свободно
мы записали true
мы захватили lock
```

Если вернуло `true`:

```
раньше было занято
мы снова записали true
кто-то другой держит lock
крутимся дальше
```

`unlock`:

```
state_.store(false);
```

освобождает lock.

22. Почему `exchange`-spinlock корректен

Главное свойство:

```
только один поток может увидеть старое значение false
```

Потому что `exchange(true)` атомарен.

Если много потоков одновременно делают:

```
state_.exchange(true)
```

то операции выстроятся в некоторый порядок.

Первый, кто увидел `false`, станет владельцем.

Остальные увидят `true` и продолжат ждать.

Так выполняется safety:

```
не более одного потока в critical section
```

23. Проблемы простого spinlock

Проблема 1. Жжёт CPU

Поток постоянно выполняет цикл:

```
while (state_.exchange(true)) {}
```

Он не спит и занимает ядро.

Если lock держат долго, это плохо.

Проблема 2. Плохо при oversubscription

Oversubscription — потоков больше, чем ядер.

Сценарий:

```
поток A держит lock, но его вытеснили с CPU  
потоки B, C, D крутятся в spinlock и занимают CPU  
A не получает CPU и не может освободить lock
```

Получается очень плохо: ждущие потоки мешают владельцу освободить блокировку.

Проблема 3. Cache ping-pong

`exchange` — это запись.

Если много потоков крутятся на:

```
state_.exchange(true)
```

каждый постоянно пишет в одну и ту же cache line.

Это вызывает invalidation кэш-линии у других ядер.

Результат:

```
cache line бегает между ядрами  
много трафика когерентности  
плохая масштабируемость
```

Проблема 4. Несправедливость

Простой spinlock не гарантирует FIFO.

Когда lock освободился, победит тот, кто первым выполнит `exchange`.

Один поток может долго проигрывать.

Это называется:

```
starvation
```

24. Когда spinlock уместен

Spinlock может быть хорош, если:

критическая секция очень короткая
lock держат десятки/сотни наносекунд
нет долгих I/O внутри
потоков не сильно больше, чем ядер
важна минимальная latency

Например:

низкоуровневые структуры
короткие участки внутри runtime
kernel / embedded / low-latency systems

Spinlock плох, если:

критическая секция долгая
внутри может быть I/O
поток может заснуть
много потоков конкурируют за lock
потоков больше, чем ядер

25. `pause`, `yield`, `sleep`

Если поток крутится, можно улучшить ожидание.

`pause`

На x86 есть инструкция `pause`.

В C++ через intrinsic:

```
#include <immintrin.h>

_mm_pause();
```

Она говорит процессору:

я нахожусь в spin-loop

Плюсы:

меньше энергопотребление
лучше для hyper-threading
меньше нагрузка на pipeline

Это не системный вызов.

`yield`

```
std::this_thread::yield();
```

Поток говорит планировщику:

я готов уступить CPU другому потоку

Это может быть системный вызов или дорогостоящая операция.

Подходит, если ожидание затянулось.

`sleep`

```
std::this_thread::sleep_for(1ms);
```

Слишком грубо для spinlock.

Можно проспать момент освобождения lock-а и получить плохую latency.

26. Test-and-test-and-set spinlock

Простой spinlock:

```
while (state_.exchange(true)) {}
```

плох тем, что каждый ожидающий поток постоянно делает RMW-запись.

Можно улучшить:

```
void lock() {  
    while (true) {  
        while (state_.load(std::memory_order_relaxed)) {  
            _mm_pause();  
        }  
    }  
}
```

```

    }

    if (!state_.exchange(true, std::memory_order_acquire)) {
        return;
    }
}
}

```

Идея:

пока lock занят, крутимся на обычном load
 load не инвалидирует cache line
 только когда увидели свободно, пробуем exchange

Это называется:

TTAS – test-and-test-and-set

Плюс:

меньше cache ping-pong
 меньше RMW-операций
 лучше под конкуренцией

27. Memory order для spinlock

По умолчанию атомики используют:

`std::memory_order_seq_cst`

Это самый строгий порядок.

Но для spinlock обычно достаточно:

```

exchange(true, std::memory_order_acquire)
store(false, std::memory_order_release)

```

Код:

```

class SpinLock {
public:
    void lock() {

```

```

        while (state_.exchange(true, std::memory_order_acquire)) {
            while (state_.load(std::memory_order_relaxed)) {
                // pause
            }
        }
    }

    void unlock() {
        state_.store(false, std::memory_order_release);
    }

private:
    std::atomic<bool> state_{false};
};

```

Смысл:

acquire на lock

операции после lock не могут переехать до lock
поток, захвативший lock, увидит данные,
которые предыдущий владелец записал до unlock

release на unlock

операции внутри critical section не могут переехать после unlock
изменения становятся видимыми следующему владельцу lock

Полный разбор `memory_order` будет позже. Здесь достаточно сказать:

Для реализации lock-а нужен acquire на захвате и release на освобождении.

28. `std::atomic_flag`

В стандарте есть специальный минимальный атомарный флаг:

```
std::atomic_flag flag = ATOMIC_FLAG_INIT;
```

Он поддерживает:

```
flag.test_and_set()
flag.clear()
```

Spinlock через `atomic_flag`:


```

class SpinLock {
public:
    void lock() {
        while (flag_.test_and_set(std::memory_order_acquire)) {
            // spin
        }
    }

    void unlock() {
        flag_.clear(std::memory_order_release);
    }

private:
    std::atomic_flag flag_ = ATOMIC_FLAG_INIT;
};

```

`test_and_set`:

устанавливает flag в true
возвращает старое значение

Это ровно то, что нужно для spinlock.

29. Backoff

При сильной конкуренции потоки могут мешать друг другу постоянными попытками захвата.

Идея backoff:

после неудачной попытки немного подождать
при повторных неудачах увеличивать паузу

Например:

pause 4 раза
pause 8 раз
pause 16 раз
...
потом yield

Это снижает contention.

Но конкретные числа — магические константы, которые зависят от железа, ОС и нагрузки.

30. Ticket lock

Простой spinlock несправедлив.

Ticket lock делает FIFO-порядок.

Идея:

```
next_ticket – какой билет выдаём следующему потоку
serving     – какой билет сейчас обслуживается
```

Алгоритм:

```
my_ticket = next_ticket.fetch_add(1);

while (serving.load() != my_ticket) {
    spin;
}
```

`unlock`:

```
serving.fetch_add(1);
```

Схема:

```
thread A получил билет 10
thread B получил билет 11
thread C получил билет 12

сейчас serving = 10
A входит
потом serving = 11
B входит
потом serving = 12
C входит
```

Плюс:

```
fairness / FIFO
нет starvation
```

Минус:

все потоки всё равно читают одну переменную `serving`
при каждом `unlock` все видят изменение

Для билета достаточно знать как идею справедливого `spinlock`.

31. MCS-lock

MCS-lock — более масштабируемый справедливый `lock`.

Идея:

потоки образуют очередь узлов
каждый поток спинит на своём локальном флаге
предыдущий поток будит следующего

Плюс:

меньше `cache bouncing`
хорошо масштабируется на NUMA
FIFO fairness

Это уже профессиональное дополнение. В базовом ответе достаточно упомянуть:

Для высоких нагрузок существуют очередные `spinlock`-и вроде `ticket lock` и `MCS-lock`.

32. Реализация `mutex` через `spinlock`

Можно сделать свой примитив:

```
class Mutex {
public:
    void lock() {
        spin_.lock();
    }

    void unlock() {
        spin_.unlock();
    }

private:
    SpinLock spin_;
};
```

Но по сути spinlock уже является mutex-ом с активным ожиданием.

Более полный вариант:

```
class SpinMutex {
public:
    void lock() {
        while (state_.exchange(true, std::memory_order_acquire)) {
            while (state_.load(std::memory_order_relaxed)) {
                // optionally _mm_pause()
            }
        }
    }

    void unlock() {
        state_.store(false, std::memory_order_release);
    }

private:
    std::atomic<bool> state_{false};
};
```

Свойства:

lock захватывает блокировку
unlock освобождает
в critical section одновременно не больше одного потока
при занятости поток не спит, а крутится

33. Чем spinlock отличается от `std::mutex`

Характеристика	Spinlock	<code>std::mutex</code>
Ожидание	активное, busy wait	обычно блокирующее/парковка
CPU при ожидании	тратится	почти не тратится
Системный вызов	обычно нет	при конкуренции может быть futex/ syscall
Latency при коротком ожидании	очень низкая	выше
Длинные секции	плохо	лучше
I/O внутри critical section	очень плохо	тоже плохо, но не жжёт CPU
Oversubscription	очень плохо	лучше

Характеристика	Spinlock	<code>std::mutex</code>
Реализация	атомики	атомики + futex/ядро

Важно:

Spinlock — это не “быстрый mutex на все случаи”. Он быстрый только для очень коротких секций и правильных условий.

34. `is_lock_free`

Не все `std::atomic<T>` гарантированно реализованы без lock-a.

Для маленьких типов вроде `int`, указателей, `bool` обычно всё lock-free.

Но для крупных структур атомик может внутри использовать скрытый mutex.

Можно проверить:

```
std::atomic<long long> x;

if (x.is_lock_free()) {
    // lock-free на этой платформе
}
```

Или compile-time:

```
std::atomic<T>::is_always_lock_free
```

Для экзамена полезно сказать:

`std::atomic` не всегда означает lock-free для любого `T`.

35. Типичные ошибки

Ошибка 1. Думать, что `volatile` решает многопоточность

`volatile` не делает операции атомарными и не создаёт синхронизацию между потоками.

Плохо:

```
volatile bool flag = false;
```

Для потоков нужен:

```
std::atomic<bool> flag;
```

Ошибка 2. Использовать обычный bool для spinlock

```
bool state = false;
```

Это data race и нарушение mutual exclusion.

Нужен атомик.

Ошибка 3. Считать ++counter атомарным

++counter для обычного int / size_t — read-modify-write из нескольких шагов.

Ошибка 4. Держать spinlock долго

Внутри spinlock нельзя делать:

```
сетевой I/O  
sleep  
долгие вычисления  
ожидание другого lock-a
```

Ошибка 5. Забыть про memory order

Для lock-a нужен acquire/release.

Если использовать атомики неправильно, можно получить взаимное исключение по флагу, но некорректную видимость данных внутри critical section.

Ошибка 6. Думать, что компиляторный барьер чинит store buffering

```
asm volatile("" ::: "memory")
```

 — только барьер для компилятора.

Store buffer процессора он не очищает.

36. Что сказать на экзамене: короткая версия

Можно отвечать так:

Data race — это несинхронизированный доступ нескольких потоков к одной области памяти, если хотя бы один доступ является записью. В C++ это undefined behavior. Например, `++counter` из нескольких потоков некорректен, потому что он раскладывается на load, add и store, и обновления могут потеряться.

Для простых общих значений можно использовать `std::atomic`. Он предоставляет атомарные операции: `load`, `store`, `exchange`, `fetch_add`, `compare_exchange_weak/strong`. Операции вроде `exchange`, `fetch_add` и CAS являются read-modify-write: они атомарно читают, изменяют и записывают значение.

Store buffering — аппаратный эффект, когда запись сначала попадает в буфер записи текущего ядра и не сразу видна другим ядрам. Поэтому в тесте `x = 1; a = y` и `y = 1; b = x` возможен результат `a == 0 && b == 0`: оба потока не увидели запись другого. Компиляторный барьер это не лечит, нужен аппаратный барьер или атомики с достаточно строгим memory order.

Spinlock — это блокировка на атомике, где поток при неудаче не засыпает, а крутится в цикле. Простейшая реализация использует `std::atomic<bool>::exchange(true)` в `lock` и `store(false)` в `unlock`. Только один поток может увидеть старое значение `false`, поэтому mutual exclusion выполняется.

Минус spinlock-a — он жжёт CPU, плохо ведёт себя при долгом ожидании и при большом числе потоков. Поэтому он подходит только для очень коротких критических секций. Для улучшения используют `pause`, test-and-test-and-set, backoff, ticket lock или MCS-lock. Для обычного кода чаще используют `std::mutex`, который при конкуренции может парковать поток через futex.

37. Мини-код для ответа на доске

Счётчик через `std::atomic`

```
std::atomic<size_t> counter{0};

void worker(size_t iterations) {
    for (size_t i = 0; i < iterations; ++i) {
        counter.fetch_add(1, std::memory_order_relaxed);
    }
}
```

Для простого счётчика статистики `relaxed` достаточно, если нам не нужна синхронизация с другими данными.

Spinlock через `atomic<bool>`

```
class SpinLock {
public:
    void lock() {
        while (state_.exchange(true, std::memory_order_acquire)) {
            while (state_.load(std::memory_order_relaxed)) {
                // optional: _mm_pause();
            }
        }
    }

    void unlock() {
        state_.store(false, std::memory_order_release);
    }

private:
    std::atomic<bool> state_{false};
};
```

Что объяснить:

`exchange(true)` атомарно ставит `state = true` и возвращает старое значение
если старое `false` — lock был свободен, мы его захватили
если старое `true` — lock занят, ждём
`load` в inner loop уменьшает cache ping-pong
`acquire/release` обеспечивают видимость данных critical section

38. Мини-проверка

Теперь короче — 12 вопросов.

1. Что такое data race?
2. Почему `++counter` из нескольких потоков без синхронизации некорректен?
3. Чем data race отличается от race condition?
4. Что делает `std::atomic`?
5. Какие основные методы есть у `std::atomic`?
6. Что такое RMW-операция?
7. Что делает `compare_exchange_weak`, и почему его обычно используют в цикле?
8. Что такое store buffering и почему возможен результат `(0,0)`?
9. Почему компиляторный барьер не исправляет store buffering?
10. Почему наивный spinlock через обычный `bool` ломается?
11. Как работает spinlock через `atomic.exchange(true)`?
12. Почему spinlock не подходит для долгих критических секций?